

Python Modules

Module in Python

A **module** in Python is simply a **file containing Python code** — variables, functions, classes — that you can **reuse in other Python programs**.

File with .py extension is treated as a module.

Use of Modules

- Reusability: Write once, use anywhere.
- Organization: Keep code modular and manageable.
- Avoid Code Duplication: Common code can be placed in a module.
- Easier Maintenance & Debugging

Types of Modules

Type	Description	Example
Built-in	Pre-installed with Python	math, os, sys
User-defined	Created by users as .py files	Marvellous.py
Third-party	Installed via pip from external packages	numpy, pandas, flask

Creating and Using a User-Defined Module

Step 1: Create a Module

👉 File: Marvellous.py

```
def Display():  
    print("Jai Ganesh..")
```

```
PI = 3.14159
```

Step 2: Import and Use in Another File

👉 File: main.py

```
import Marvellous
```

```
Marvellous.Display()  
print("Value of PI:", Marvellous.PI)
```

Output:

```
Jai Ganesh..  
Value of pi: 3.14159
```

This is **how we import and use our own code written in other files**.

Ways to Import Modules

Syntax	Usage
<code>import module_name</code>	Imports whole module
<code>from module_name import x</code>	Imports specific object/function
<code>from module_name import *</code>	Imports everything (not recommended)
<code>import module_name as alias</code>	Imports with alias name for shorter usage

Example:

```
from Marvellous import Display
```

```
Display()
```

Built-in Modules

Example with math:

```
import math
```

```
print(math.sqrt(16))      # Output: 4.0
print(math.pi)            # Output: 3.141592653589793
```

Third-party Module

Example with numpy (after installation):

```
pip install numpy
```

```
import numpy as np
```

```
a = np.array([1, 2, 3])
print(a * 2)
```

__name__ and __main__ in Python

In every Python module, there is a special built-in variable called `__name__`.

- When a Python file is run **directly**, the value of `__name__` is set to `"__main__"`.
- When a Python file is **imported as a module**, the value of `__name__` is set to the **module's file name** (without `.py`).

Use of if `__name__ == "__main__"`:

To **control the execution** of certain parts of your code. It allows you to write:

- Code that runs **only when the file is executed directly**
- Code that does **not run** when the file is **imported** in another module

Example:

File: Marvellous_Module.py

```
def Display():  
    print("Welcome from Marvellous Infosystems!")  
  
print("This line always runs.")  
print("Value of __name__ is:", __name__)  
  
if __name__ == "__main__":  
    print("This code runs only when run directly.")  
    Display()
```

Scenario 1: Run directly

```
python Marvellous_Module.py
```

Output:

```
This line always runs.  
Value of __name__ is: __main__  
This code runs only when run directly.  
Welcome from Marvellous Infosystems!
```

Scenario 2: Import into another file

```
# main.py  
import module_demo
```

Output:

```
This line always runs.  
Value of __name__ is: module_demo
```

Display() is not called, because the if `__name__ == "__main__"` block is skipped.